

Lab 4

Advanced Generic Systems & Exception Safety

1. Problem Overview

You are tasked with programming a **robotic warehouse controller**. The warehouse stores **Boxes** of various data types (integers, characters, or floats). The controller must track physical constraints such as *total weight* and *robot energy*.

Critical Requirement — Atomicity: If a bulk operation fails midway (e.g. the robot runs out of energy while moving multiple boxes), the system must *roll back* to its original state so that no inventory is lost or misplaced.

2. Structural Requirements

2.1. The `Box<T>` Template

Create a simple templated structure to represent a warehouse item:

- `T data` — the actual value being stored.
- `int weight` — the physical weight of the box.

2.2. The `Storage<T>` Class

Implement a templated stack as the raw storage medium. The `pop` operation must return the entire `Box` structure so the `Warehouse` can track weight correctly.

- `void push(T item, int weight)` — creates a `Box` and pushes it.
- `Box<T> pop()` — removes and returns the top `Box`; throws an exception if empty.
- `int currentSize()` — returns the number of items currently stored.

2.3. The `Warehouse<T>` Class

This class manages the logic and physical constraints of the system.

Attributes: `maxCapacity` (item count limit), `maxWeight` (total weight limit), `currentEnergy`, `currentWeight` (running total weight of all items currently stored).

Operation Logic & Precedence:

Dispatch — Single or Bulk

Step 1. Check Energy First. Ensure $energy \geq 2$. If not, throw `PowerFailure`.

Step 2. Check Inventory Second. Ensure storage is not empty. If empty, throw `OutOfStock`.

Step 3. Execute. Decrement energy by 2, pop the item, and update total weight.

- **Receive:** Zero energy cost. Must verify `Capacity` first, then `MaxWeight`, before pushing. If both limits are violated, `WarehouseFull` takes precedence over `Overloaded`.
- **Recharge:** Adds to `currentEnergy`. There is *no* maximum energy cap. Produces no output.

3. Exception Specifications

Define a custom exception hierarchy with the triggering conditions and output strings shown below.

Exception Class	Triggering Condition	Output
PowerFailure	Dispatch attempted when $energy < 2$	ERR_POWER
WarehouseFull	Item added when $size = maxCapacity$	ERR_FULL
Overloaded	Adding item would exceed $maxWeight$ limit	ERR_WEIGHT
OutofStock	Dispatch attempted on empty storage	ERR_EMPTY

Table 1: Exception classes, triggering conditions, and required output strings.

4. The Challenge: Atomic Bulk Dispatch

Implement `void bulkDispatch(int k):`

1. The function attempts to remove k items iteratively, applying the standard Dispatch checks (**energy first**, then **empty check**) for each item.
2. **No output during processing.** Store removed items in a temporary buffer.
3. **On failure at the i^{th} removal:**
 - Catch the exception (`PowerFailure` or `OutofStock`).
 - **Rollback:** push all temporarily removed items back *in reverse order* to restore the original stack state.
 - Restore `currentEnergy` to its value at the start of the call.
 - Restore `currentWeight` by adding back the weights of all re-pushed items.

- Re-throw the exception to the caller.
4. **On success:** print the data of each removed item on a new line, in the order they were removed.

Boundary condition: You may assume $k \geq 1$ for all `bulkDispatch` calls. A call with $k = 0$ will not appear in any test input.

5. Input / Output Format

5.1. Input Specification

Line 1 — Warehouse configuration (four space-separated values):

- **Capacity** (`int`) — maximum item count.
- **MaxWeight** (`int`) — maximum total weight.
- **InitialEnergy** (`int`) — starting energy units.
- **Type** (`char`) — item data type: `I` (int), `C` (char), `F` (float).

Line 2 — integer N : total number of operations.

Subsequent lines — one operation per line:

Command	Meaning
1 <data> <weight>	Add item (push to storage)
2	Dispatch item (pop and print data)
3 <value>	Recharge energy
4 <k>	Atomic bulk dispatch of k items

5.2. Sample 1

Input

```
2 100 3 I
4
1 10 60
1 20 50
2
4 2
```

Expected Output

```
ERR_WEIGHT
10
ERR_POWER
```

Explanation:

Command	What happens	State after
1 10 60	Success. Item added.	Wt:60 En:3 Sz:1
1 20 50	Fails. $60 + 50 = 110 > 100$. Throws Overloaded.	ERR_WEIGHT
2	Success. Item 10 popped; energy 3 → 1.	Wt:0 En:1 Sz:0
4 2	Fails. Energy $1 < 2$; PowerFailure on first item. No rollback needed.	ERR_POWER

5.3. Sample 2 — Rollback Test

This test case specifically exercises the **atomic rollback** guarantee of `bulkDispatch`. The bulk operation partially succeeds before hitting a `PowerFailure`, at which point the stack and energy must be fully restored.

Input

```
4 200 6 I
7
1 10 40
1 20 50
```

```
1 30 30
4 4
1 99 60
4 2
2
```

Expected Output

```
ERR_POWER
99
30
20
```

Explanation:

Command	What happens	State after
1 10 40	Success. Stack: [10].	Wt:40 En:6 Sz:1
1 20 50	Success. Stack: [20, 10].	Wt:90 En:6 Sz:2
1 30 30	Success. Stack: [30, 20, 10].	Wt:120 En:6 Sz:3

Command	What happens	State after
4 4	Save state: energy= 6. Pops 30 (en 6 $\rightarrow$ 4), 20 (en 4 $\rightarrow$ 2), 10 (en 2 $\rightarrow$ 0). 4th check: $0 < 2$ — PowerFailure . Rollback : re-push 10, then 20, then 30; restore energy to 6; restore weight to 120. Stack: [30, 20, 10].	ERR_POWER
1 99 60	Success . $120 + 60 = 180 \leq 200$. Stack: [99, 30, 20, 10].	Wt:180 En:6 Sz:4
4 2	Success . Pops 99 (en 6 $\rightarrow$ 4), then 30 (en 4 $\rightarrow$ 2). Prints 99, then 30. Stack: [20, 10].	Wt:90 En:2 Sz:2
2	Success . Pops 20; energy $2 \rightarrow 0$. Prints 20. Stack: [10].	Wt:40 En:0 Sz:1

6. Submission Guidelines

- You must implement the **Storage** stack manually using dynamic arrays or linked nodes. **Use of STL is strictly forbidden.**
- Ensure all class templates and member functions are fully defined.

- **Exception handling must use `try-catch` blocks.** The following rules are enforced:
 - `bulkDispatch` must use a `try-catch` block internally to intercept failures, perform rollback, and re-throw — **not** pre-check conditions with `if` statements to avoid throwing.
 - The `main` input loop must wrap each operation in its own `try-catch` block that catches and prints the appropriate error string.
 - You should **not** use conditional checks as a substitute for catching exceptions (e.g. checking `currentSize() == 0` before calling `pop()` to avoid the `OutOfStock` throw is forbidden — `pop()` must be allowed to throw and the exception must be caught). **Doing this will result in 0 marks.**
- Implement destructors to prevent memory leaks.